

МИНИСТЕРСТВО ЦИФРОВОГО РАЗВИТИЯ, СВЯЗИ И МАССОВЫХ
КОММУНИКАЦИЙ
РОССИЙСКОЙ ФЕДЕРАЦИИ

Федеральное государственное бюджетное образовательное учреждение
высшего образования

"Сибирский государственный университет телекоммуникаций и
информатики"

Кафедра телекоммуникационных систем и вычислительных средств
(ТС и ВС)

ОТЧЕТ № 14
по дисциплине
"Web-технологии"

по теме:
СКАЧИВАНИЕ ФАЙЛА В ФОРМАТЕ PDF. ЭФФЕКТ
MATERIAL WAVE

Студент:
Группа ИКС-531

Д.А. Язиков

Преподаватель:
ст. преподаватель

А.В. Андреев

Новосибирск 2026 г.

СОДЕРЖАНИЕ

ВВЕДЕНИЕ	4
1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ.....	6
1.1 Семантическая разметка HTML5	6
1.2 Редактируемый контент: атрибут <code>contenteditable</code>	6
1.3 Клиентское хранилище <code>localStorage</code>	7
1.4 Чтение файлов: <code>FileReader</code> и <code>canvas</code>	7
1.5 Печать страницы: <code>window.print()</code> и <code>@media print</code>	8
1.6 CSS-анимации.....	8
1.7 Эффект Material Wave (ripple)	9
2 ОПИСАНИЕ ПРОЕКТА.....	10
2.1 Технологический стек.....	10
2.2 Архитектура приложения.....	10
2.3 Конфиденциальность данных	11
2.4 Структура каталога <code>lab14/</code>	11
2.5 Способ публикации	12
3 ХОД РАБОТЫ	13
3.1 Каркас страницы.....	13
3.2 Рендер резюме из конфигурации	14
3.3 Редактирование и сохранение	15
3.4 Загрузка фотографии	16
3.5 CSS-анимации изменений	17
3.6 Эффект Material Wave	17
3.7 Экспорт в PDF	18
3.8 Адаптивность	19
3.9 Публикация на сервере.....	19
ЗАКЛЮЧЕНИЕ	20
А ВОССТАНОВЛЕННЫЕ ИЛЛЮСТРАЦИИ.....	22

Примечание к архивной копии

Это архивная версия сохранившегося текста отчёта, а не повторно выполненная лабораторная работа. Исторические скриншоты отсутствуют, поэтому соответствующие рисунки в этой копии опущены. Иллюстрации в приложении получены повторным отображением сохранённых исходников и не заменяют оригинальных свидетельств выполнения. Сохранились HTML/CSS/JS-файлы резюме. Повторные рендеры не доказывают публикацию на прежнем домене или прежнее состояние Docker-стенда.

ВВЕДЕНИЕ

Современная веб-страница --- это не статический документ, а полноценное клиентское приложение. Пользователь ожидает, что интерфейс реагирует на действия мгновенно, введённые данные не теряются при перезагрузке, а результат работы можно сохранить в привычном формате. Все эти возможности реализуются тремя базовыми технологиями браузера: **HTML** описывает структуру, **CSS** --- оформление и анимации, **JavaScript** --- поведение и работу с данными.

Предыдущая лабораторная работа (№ 13) была посвящена основам JavaScript и построению сайта на данных внешнего API. Текущая работа развивает практические навыки клиентской разработки на примере *интерактивного резюме*: страница не просто отображает информацию, а позволяет редактировать её прямо в браузере, сохраняет правки между сессиями и экспортирует готовый документ в формат PDF.

В качестве темы выбрана веб-страница с резюме, свёрстанная по типовому шаблону. Каждый текстовый элемент резюме редактируется <<на месте>> (атрибут `contenteditable`), все изменения сохраняются в хранилище браузера `localStorage` и восстанавливаются при следующем открытии. Кнопка <<Скачать PDF>> формирует документ средствами самого браузера. Любое изменение элемента сопровождается CSS-анимацией, а клики по элементам --- эффектом **Material Wave** (<<волна>>, ripple) из дизайн-системы Google Material Design. Готовое приложение публикуется на собственном домашнем сервере под обратным прокси **Caddy** и доступно по адресу <https://resume.server34.netcraze.club>.

Согласно требованиям методички, разработка ведётся **только базовыми технологиями** --- HTML, CSS и JavaScript --- без сторонних библиотек и фреймворков и без этапа сборки.

Цель работы --- получить практические навыки разработки интерактивных веб-приложений: реализовать редактирование контента, сохранение данных на стороне клиента, экспорт страницы в PDF, CSS-анимации и адаптивную вёрстку.

Задачи:

1. Сверстать веб-страницу с резюме по типовому шаблону, используя семантическую разметку HTML5.
2. Реализовать редактирование текстовых элементов резюме с сохранением изменений внутри самих элементов.
3. Добавить кнопку <<Скачать>>, преобразующую текущую версию резюме в формат PDF и начинающую загрузку файла.
4. Анимировать изменения элементов средствами CSS-анимаций.
5. Обеспечить адаптивность страницы и эффект Material Wave при кликах.
6. Реализовать сохранение введённых данных при перезагрузке страницы и опубликовать приложение на публичном хостинге.

Стенд:

- современный браузер (Chrome, Firefox, Safari) --- среда исполнения приложения; страница открывается как напрямую через `file://`, так и с веб-сервера;
- редактор кода с линтерами JSHint/ESLint --- проверка качества JavaScript-кода;
- валидатор разметки `validator.w3.org` --- проверка семантики вёрстки;
- собственный домашний сервер (Debian 12, Docker Compose) с обратным прокси Caddy и автоматическим выпуском TLS-сертификатов Let's Encrypt --- площадка для публикации готового сайта.

1 ТЕОРЕТИЧЕСКИЕ СВЕДЕНИЯ

1.1 Семантическая разметка HTML5

HTML5 вводит набор *семантических* элементов, которые описывают не оформление, а смысл блока: `<header>` (шапка), `<main>` (основное содержимое), `<aside>` (вспомогательная информация), `<section>` (раздел), `<article>` (самостоятельная единица контента), `<nav>`, `<footer>`. Семантика улучшает доступность (скринридеры понимают структуру), индексацию поисковыми системами и читаемость кода.

Корректность разметки проверяется **валидатором W3C** (<https://validator.w3.org>): он выявляет незакрытые теги, недопустимое вложение элементов, отсутствие обязательных атрибутов (например, `alt` у изображения). Для резюме семантическая структура естественна: `<article>` --- всё резюме, `<aside>` --- боковая колонка с контактами и навыками, `<section>` --- смысловые разделы (опыт, образование), `<h1>`--`<h3>` --- иерархия заголовков.

1.2 Редактируемый контент: атрибут `contenteditable`

Глобальный атрибут `contenteditable="true"` превращает любой элемент в редактируемое поле: пользователь кликает по тексту и правит его прямо на странице, без отдельных `<input>` или `<textarea>`. Изменённый текст остаётся *внутри самого элемента* --- это удобно для интерфейсов-конструкторов, где форма и результат совпадают.

```
<h1 contenteditable="true" data-field="nameИмя"> Фамилия</h1>
```

Listing 1.1 — Редактируемый элемент

При работе с `contenteditable` учитываются особенности:

- нажатие `Enter` по умолчанию вставляет блочный элемент (`<div>` или `<br>`) --- в однострочных полях это поведение перехватывается и отменяется;
- вставка из буфера (`paste`) может принести чужую разметку и стили
--- содержимое буфера очищается до простого текста;

-- опустевшее поле сохраняет служебный `<br>`, мешающий показу подсказки-плейсхолдера через псевдокласс `:empty`.

1.3 Клиентское хранилище `localStorage`

Web Storage API предоставляет объект `localStorage` --- постоянное хранилище пар <<ключ--значение>> в браузере. В отличие от `sessionStorage`, данные не стираются при закрытии вкладки. Объём --- около 5 МБ на источник (`origin`). Хранятся только строки, поэтому объекты сериализуются через `JSON.stringify` и читаются через `JSON.parse`.

```
localStorage.setItem('resume', JSON.stringify(data));  
const data = JSON.parse(localStorage.getItem('resume'));
```

Listing 1.2 — Сохранение и чтение данных

Доступ к `localStorage` может быть запрещён (приватный режим, настройки браузера) --- вызовы оборачиваются в `try/catch`, при сбое приложение продолжает работу без сохранения. Все данные `localStorage` не покидают браузер пользователя --- это естественный способ хранить личную информацию, не передавая её на сервер.

1.4 Чтение файлов: `FileReader` и `canvas`

Элемент `<input type="file">` даёт доступ к выбранному пользователем файлу. Объект **`FileReader`** читает файл асинхронно; метод `readAsDataURL` возвращает содержимое в виде *data-URL* --- строки `data:image/...;base64, ...`, которую можно подставить в атрибут `src` изображения.

Чтобы фотография не занимала лишнее место в `localStorage`, она уменьшается через элемент `<canvas>`: изображение рисуется на холсте меньшего размера методом `drawImage`, а `canvas.toDataURL` отдаёт сжатый JPEG.

1.5 Печать страницы: `window.print()` и `@media print`

Метод `window.print()` открывает системный диалог печати. Современные браузеры предлагают в нём виртуальный принтер <<Сохранить как PDF>>, что позволяет экспортировать страницу в PDF **без единой сторонней библиотеки**. Имя сохраняемого файла берётся из `document.title`.

Внешний вид печатной версии задаётся медиавыражением `@media print`: в нём скрываются интерфейсные элементы (панель управления, подсказки), убираются тени и фоновые подложки, а правило `break-inside: avoid` запрещает разрывать смысловые блоки между страницами. Так из одной и той же разметки получаются и экранная, и печатная версии документа.

```
@media print {  
  .toolbar { display: none; }  
  .entry { break-inside: avoid; }  
}
```

Listing 1.3 — Правила только для печати

1.6 CSS-анимации

CSS-анимация описывается правилом `@keyframes`, задающим ключевые кадры (значения свойств в процентах времени), и свойством `animation`, которое привязывает набор кадров к элементу и задаёт длительность, функцию плавности и число повторов. Анимация выполняется браузером и не требует JavaScript --- скрипт лишь добавляет или снимает CSS-класс, запускающий нужный эффект.

```
@keyframes saved-flash {  
  0% { background-color: rgba(34, 197, 94, 0.45); }  
  100% { background-color: transparent; }  
}  
.is-saved { animation: saved-flash 420ms ease-out; }
```

Listing 1.4 — Описание и применение анимации

Медиавыражение `@media (prefers-reduced-motion: reduce)` позволяет отключить анимации для пользователей, чувствительных к движению.

1.7 Эффект Material Wave (ripple)

Material Wave, или *ripple* (<<волна>>), --- характерный приём дизайн-системы Google **Material Design**: при клике из точки касания расходится круговая волна, дающая визуальный отклик на действие. Эффект реализуется без библиотек: по событию `pointerdown` скрипт вычисляет координаты клика относительно элемента, создаёт `<span>` нужного диаметра и помещает его в точку касания; CSS-анимация увеличивает круг и плавно уводит его в прозрачность, после чего `<span>` удаляется. Элемент-хост получает свойства `position: relative` и `overflow: hidden`, чтобы волна не выходила за его границы.

2 ОПИСАНИЕ ПРОЕКТА

2.1 Технологический стек

Приложение построено **исключительно базовыми технологиями** браузера, без сборщиков, транспиляторов и менеджеров зависимостей --- этого требует методичка, и это же подчёркивает: для интерактивной страницы не нужен <<ритуал>> современного фронтенда.

- **HTML5** --- семантическая разметка резюме и панели управления;
- **CSS3** --- двухколоночная вёрстка на CSS Grid, анимации (@keyframes), адаптивность (@media) и отдельная печатная версия (@media print);
- **Vanilla JavaScript** (ES2024) --- рендер резюме, работа с localStorage, чтение фотографии, эффект Material Wave; классические теги <script> без модулей, поэтому страница работает и при открытии напрямую через file://.

Никаких npm install и node_modules: проект --- это набор статических файлов, которые браузер исполняет как есть.

2.2 Архитектура приложения

Резюме не свёрстано вручную в index.html, а *собирается из данных*. Это разделяет содержимое и разметку и упрощает заполнение резюме.

```
config.js (RESUME_DATA)                                index.html пустой( каркас)
    |                                                         |
    +-----> renderResume(data) -----+
                |
                v
        applyFields(localStorage) <-- правкипользователя
                |
                vготовоерезюмеv
                DOM
```

Listing 2.1 — Поток данных при загрузке страницы

Объект `RESUME_DATA` из файла `config.js` описывает резюме: имя, должность, контакты, навыки, языки, опыт, образование. Функция `renderResume` строит из него разметку и вставляет в страницу. Затем `applyFields` накладывает поверх сохранённые в `localStorage` правки пользователя, если они есть.

Файлы JavaScript разделены по образцу учебного шаблона курса:

- `config.js` --- только данные (объект `RESUME_DATA`);
- `resume.js` --- чистые функции: рендер, работа с хранилищем, чтение фотографии, создание <<волны>>, запуск анимаций;
- `start.js` --- оркестрация: функция `start()` навешивает обработчики событий по `DOMContentLoaded`.

2.3 Конфиденциальность данных

Репозиторий публичный, поэтому в файл `config.js` помещён **выдуманный** персонаж --- фронтенд-разработчик с вымышленными ФИО, контактами и нейтральным аватаром. Реальное резюме заполняется иначе: пользователь открывает страницу и редактирует текст прямо в браузере, а фотографию загружает кликом по аватару. Все реальные данные и фото хранятся **только в `localStorage` браузера** и ни в один файл проекта не записываются --- то есть физически не могут попасть в систему контроля версий. Кнопка <<Сбросить>> очищает хранилище и возвращает страницу к выдуманному шаблону.

2.4 Структура каталога `lab14/`

```
lab14/ |—
  README.md |—
  deploy.sh      # деплойснос/ сайтаза Caddy |—
  resume/        # сайтрезюме- |—
    index.html   # каркас + панельуправления |—
    css/style.css # вёрстка, анимации, печать, адаптив |—
    js/ ||—
      config.js  # RESUME_DATA -выдуманныйперсонаж ||—
      resume.js  # чистыефункциирендер (, storage, ripple)
                ||—
```

```

    start.js      # start() —навешиваниесобытий|  └─
    img/avatar.svg  # нейтральныйаватарпоумолчанию└─
    latex-report/  # это тотчёт

```

Listing 2.2 — Файловая структура lab14/

2.5 Способ публикации

Готовое приложение публикуется на собственном домашнем сервере (Debian 12, Docker Compose), на котором уже работает обратный прокси **Caddy** с автоматическими TLS-сертификатами Let's Encrypt. Сайт-резюме добавляется в эту инфраструктуру как ещё один Docker-контейнер на образе `nginx:alpine`, раздающий статические файлы каталога `resume/`.

```

браузер
-> 443/tcp  -> Caddy (Let's Encrypt TLS)
              |
              +--> resume:80 (nginx) -> /usr/share
                  /nginx/html

```

Listing 2.3 — Маршрут запроса до сайта

Адрес сайта --- <https://resume.server34.netcraze.club>. Деплой автоматизирован идемпотентным `bash`-скриптом `deploy.sh`; подробное описание его шагов приведено в **главе 3**. На сервере оказывается резюме с выдуманным персонажем --- личные данные пользователя остаются только в его браузере.

3 ХОД РАБОТЫ

В этой главе по порядку описана реализация интерактивного резюме: каркас страницы, рендер из конфигурации, редактирование с сохранением, загрузка фотографии, CSS-анимации, эффект Material Wave, экспорт в PDF и публикация на сервере. Для каждого этапа приведён код решения и результат.

3.1 Каркас страницы

Файл `index.html` содержит панель управления `<header>` с кнопками `<<Скачать PDF>>` и `<<Сбросить>>`, пустой контейнер `<main id="resume-root">` (его заполняет рендер) и подключение трёх скриптов. Скрипты --- классические, без `type="module"`, поэтому страница работает и при открытии напрямую через `file://`.

```
<header class="toolbar">
  <span class="toolbar__brandКонструктор"> резюме</span>
  <p class="toolbar__hintКликните"> потекстуилифото ,
    чтобыотредактировать </p>
  <div class="toolbar__actions">
    <button id="btn-pdf" class="btn btn--primary ripple-host"
      type="buttonСкачать"> PDF</button>
    <button id="btn-reset" class="btn btn--ghost ripple-host"
      type="buttonСбросить"></button>
  </div>
</header>

<main id="resume-root" class="resume-wrap"></main>
<input type="file" id="photo-input" accept="image/*" hidden>

<script src="js/config.js"></script>
<script src="js/resume.js"></script>
<script src="js/start.js"></script>
<script>
  document.addEventListener("DOMContentLoaded", function () {
    start(); });
</script>
```

Listing 3.1 — Ключевая часть `index.html`

3.2 Рендер резюме из конфигурации

Содержимое резюме задаётся объектом `RESUME_DATA` в файле `config.js`. Функция `renderResume` собирает из него семантическую разметку шаблонными строками: массивы навыков, контактов, записей опыта и образования преобразуются методом `map`. Каждому текстовому узлу присваивается атрибут `contenteditable="true"` и уникальный `data-field`, по которому поле связывается с хранилищем.

```
const expHtml = data.experience.map(function (item, i) {
  return `
    <li class="entry ripple-host">
      <div class="entry__top">
        <h3 class="entry__role editable" contenteditable=
          ="true"
            data-field="exp-${i}-role">${escapeHtml(item.
              role)}</h3>
        <span class="entry__period editable"
          contenteditable="true"
            data-field="exp-${i}-period">${escapeHtml(
              item.period)}</span>
      </div>
      <p class="entry__company editable" contenteditable="
        true"
          data-field="exp-${i}-company">${escapeHtml(item.
            company)}</p>
      <p class="entry__details editable editable--multiline
        " contenteditable="true"
          data-field="exp-${i}-details">${escapeHtml(item.
            details)}</p>
    </li>`;
}).join('');
```

Listing 3.2 — Рендер записей опыта работы (фрагмент `resume.js`)

Все значения пропускаются через функцию `escapeHtml` --- защита от XSS при сборке разметки через `innerHTML`. Резюме свёрстано в две колонки на CSS Grid: боковая колонка `<aside>` (фото, контакты, навыки, языки) и основная `<main>` (имя, <<Обо мне>>, опыт, образование).

3.3 Редактирование и сохранение

Функция `start()` навешивает на каждое поле `.editable` обработчики. По событию `focus` запускается анимация-подсветка, по `blur` --- сохранение и анимация <<сохранено>>. Сохранение происходит не на каждое нажатие клавиши, а один раз при уходе из поля; флаг `dirty` отмечает наличие несохранённых правок.

```
field.addEventListener('focus', function () {
    triggerAnimation(field, 'is-editing');
});
field.addEventListener('input', function () {
    dirty = true;
});
field.addEventListener('blur', function () {
    flushSave();
    triggerAnimation(field, 'is-saved');
});
field.addEventListener('keydown', function (event) {
    if (event.key === 'Enter' && !field.classList.contains('
        editable--multiline')) {
        event.preventDefault();
        field.blur();
    }
});
```

Listing 3.3 — Обработчики редактируемого поля (`start.js`)

Тексты всех полей собираются в объект и сохраняются в `localStorage`; при загрузке страницы они восстанавливаются. Чтение и запись обёрнуты в `try/catch` --- при недоступном хранилище приложение продолжает работу.

```
function collectFields() {
    const fields = {};
    document.querySelectorAll('[data-field]').forEach(function (
        el) {
        fields[el.getAttribute('data-field')] = el.innerText.trim
            ();
    });
    return fields;
}
```

```
function saveResumeData(data) {
  try {
    localStorage.setItem(STORAGE_KEY, JSON.stringify(data));
    return true;
  } catch (e) {
    return false;
  }
}
```

Listing 3.4 — Сбор полей и сохранение (resume.js)

Восстановление правок выполняется через `textContent` (а не `innerHTML`) --- сохранённое значение всегда вставляется как обычный текст, поэтому stored-XSS через `contenteditable` или вставку из буфера невозможен.

3.4 Загрузка фотографии

Клик по аватару открывает скрытый `<input type="file">`. Выбранный файл читается объектом `FileReader`, затем уменьшается через `<canvas>` до 400 пикселей по большей стороне и кодируется в JPEG --- так фотография укладывается в квоту `localStorage`.

```
function readPhoto(file, callback) {
  if (!file || file.type.indexOf('image/') !== 0) { return; }
  const reader = new FileReader();
  reader.onload = function () {
    const img = new Image();
    img.onload = function () {
      const maxSide = 400;
      const scale = Math.min(1, maxSide / Math.max(img.
        width, img.height));
      const canvas = document.createElement('canvas');
      canvas.width = Math.round(img.width * scale);
      canvas.height = Math.round(img.height * scale);
      canvas.getContext('2d').drawImage(img, 0, 0, canvas.
        width, canvas.height);
      callback(canvas.toDataURL('image/jpeg', 0.85));
    };
    img.src = reader.result;
  };
}
```

```
};
reader.readAsDataURL(file);
}
```

Listing 3.5 — Чтение и сжатие фотографии (resume.js)

3.5 CSS-анимации изменений

Все анимации описаны в CSS правилами `@keyframes`; JavaScript лишь переключает классы. Функция `triggerAnimation` снимает класс, делает `force-reflow` и навешивает класс заново --- так одноразовая анимация перезапускается при каждом изменении.

```
@keyframes edit-pulse {
  0%    { box-shadow: inset 0 0 0 0 rgba(59, 110, 240, 0); }
  35%   { box-shadow: inset 0 0 0 2px #3b6ef0; }
  100%  { box-shadow: inset 0 0 0 0 rgba(59, 110, 240, 0); }
}
@keyframes saved-flash {
  0%    { background-color: rgba(34, 197, 94, 0.45); }
  100%  { background-color: transparent; }
}
.is-editing { animation: edit-pulse 420ms ease-out; }
.is-saved   { animation: saved-flash 420ms ease-out; }
```

Listing 3.6 — Анимации входа в правку и сохранения (style.css)

Анимации `edit-pulse` и `saved-flash` изменяют разные свойства (`box-shadow` и `background-color`), поэтому корректно сосуществуют, если классы окажутся на элементе одновременно.

3.6 Эффект Material Wave

<<Волна>> создаётся одним делегированным обработчиком `pointerdown` на всём документе. Метод `closest('.ripple-host')` находит ближайший элемент-хост; функция `createRipple` вычисляет диаметр и положение круга и добавляет `<span>`, который удаляется по окончании анимации.

```
function createRipple(event, host) {
  const rect = host.getBoundingClientRect();
```

```

const size = Math.max(rect.width, rect.height);
const radius = size / 2;
const ripple = document.createElement('span');
ripple.className = 'ripple';
ripple.style.width = size + 'px';
ripple.style.height = size + 'px';
ripple.style.left = (event.clientX - rect.left - radius) + '
    px';
ripple.style.top = (event.clientY - rect.top - radius) + 'px
    ';
ripple.addEventListener('animationend', function () {
    ripple.remove();
}, { once: true });
host.appendChild(ripple);
}

```

Listing 3.7 — Создание <<волны>> (resume.js)

3.7 Экспорт в PDF

Кнопка <<Скачать PDF>> вызывает `window.print()`. Перед печатью имя из резюме записывается в `document.title` --- оно становится именем PDF-файла в диалоге <<Сохранить как PDF>>; после печати заголовков восстанавливается по событию `afterprint`.

```

document.getElementById('btn-pdf').addEventListener('click',
function () {
    flushSave();
    const previousTitle = document.title;
    document.title = buildPrintFilename(collectFields().name);
    window.addEventListener('afterprint', function restore() {
        document.title = previousTitle;
        window.removeEventListener('afterprint', restore);
    });
    window.print();
});

```

Listing 3.8 — Запуск печати (start.js)

Внешний вид PDF задаёт блок `@media print`: панель управления, подсказки и <<волны>> скрываются, убираются тени, блоки резюме защищаются от разрыва между страницами правилом `break-inside: avoid`.

3.8 Адаптивность

Двухколоночная сетка CSS Grid перестраивается медиавыражениями: при ширине менее 768 пикселей колонки складываются в одну, при ширине менее 480 пикселей дополнительно уменьшаются заголовки. Свойство `grid-template-areas` позволяет менять порядок колонок без правки разметки.

3.9 Публикация на сервере

Готовый сайт публикуется на собственном домашнем сервере за обратным прокси **Caddy**. В корне `lab14/` лежит идемпотентный скрипт `deploy.sh` с двумя ключами:

```
./deploy.sh --install      # развернуть сайт
./deploy.sh --uninstall    # снести сайт после показа
```

Listing 3.9 — Запуск деплоя и сноса сайта

Шаги режима `--install`:

1. Запрашивает домен (по умолчанию `--- resume.server34.netcraze.c` и путь к исходникам сайта.
2. Проверяет, что Caddy уже установлен; иначе --- ошибка и выход.
3. Делает резервные копии `docker-compose.yml` и `Caddyfile` с меткой времени.
4. Синхронизирует исходники `lab14/resume/` в `/opt/stack/resume/s` командой `rsync`.
5. Добавляет (если ещё нет) сервис `resume` на образе `nginx:alpine` в общий `docker-compose.yml`.
6. Дописывает блок поддомена в `Caddyfile`, запускает контейнер и перезагружает Caddy.

Режим `--uninstall` выполняет обратные действия: останавливает контейнер, `awk`-фильтром вырезает сервис из `docker-compose.yml` и блок поддомена из `Caddyfile`, удаляет каталог сайта и перезагружает Caddy. Первый запрос к новому поддомену запускает выпуск TLS-сертификата Let's Encrypt; готовый адрес --- <https://resume.server34.netcraze.club>.

ЗАКЛЮЧЕНИЕ

В ходе выполнения лабораторной работы 14 разработано интерактивное веб-резюме --- одностраничное приложение, реализованное **только базовыми технологиями** (HTML, CSS, JavaScript) без сторонних библиотек, фреймворков и этапа сборки.

Решены все задачи методички:

- страница с резюме свёрстана по типовому шаблону на семантической разметке HTML5 в две колонки средствами CSS Grid;
- реализовано редактирование текстовых элементов <<на месте>> через атрибут `contenteditable`; правки сохраняются внутри самих элементов и в хранилище браузера `localStorage`;
- добавлена кнопка <<Скачать>>, экспортирующая текущую версию резюме в PDF средствами `window.print()` и отдельной таблицы стилей `@media print` --- без единой сторонней библиотеки;
- все изменения элементов сопровождаются CSS-анимациями (`@keyframes`): подсветка при входе в правку и вспышка при сохранении;
- добавлен эффект **Material Wave** (ripple) при кликах и обеспечена адаптивная вёрстка для разрешений десктопа, планшета и смартфона;
- введённые данные сохраняются при перезагрузке страницы, а готовое приложение опубликовано на собственном домашнем сервере за обратным прокси **Caddy** с TLS-сертификатом Let's Encrypt по адресу <https://resume.server34.netcraze.club>.

Дополнительные приёмы, применённые в проекте сверх явных требований:

- сборка резюме из конфигурации (`config.js`) --- разделение данных и разметки;
- загрузка фотографии через `FileReader` с уменьшением размера средствами `<canvas>` --- чтобы изображение укладывалось в квоту `localStorage`;

- конфиденциальность: в публичный репозиторий попадает только выдуманный персонаж, а реальные данные и фото хранятся исключительно в браузере пользователя;
- восстановление правок через `textContent` --- структурная защита от stored-XSS;
- учёт системной настройки `prefers-reduced-motion`.

Полученные навыки --- редактируемый контент, клиентское хранилище данных, работа с файлами, генерация документов и CSS-анимации --- составляют основу разработки современных интерактивных веб-приложений и применимы как в проектах на чистом JavaScript, так и в работе с любыми фреймворками.

А ВОССТАНОВЛЕННЫЕ ИЛЛЮСТРАЦИИ

Все рисунки в этом приложении созданы при архивировании по сохранённым файлам проекта. Они не являются прежними скриншотами Figma, Codepen или удалённого сервера.

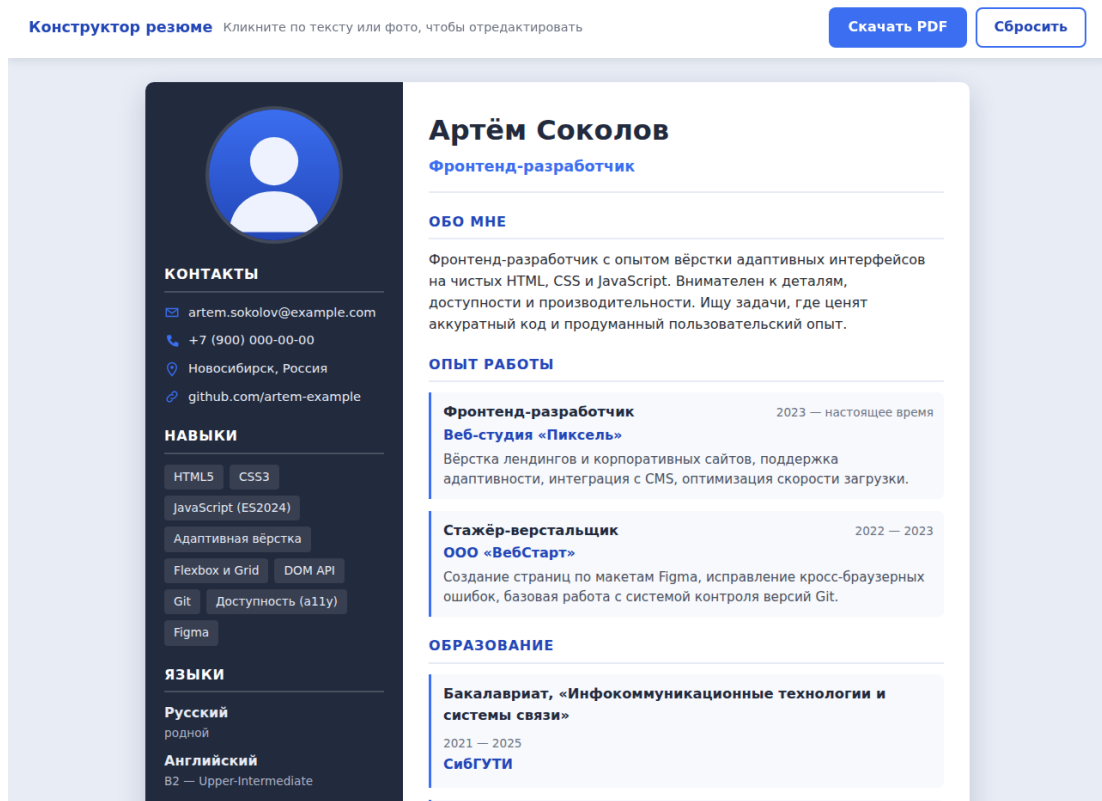


Рисунок 1 — Повторный рендер сохранённого резюме: десктоп, фрагмент

Конструктор резюме

Скачать PDF

Сбросить

Кликните по тексту или фото, чтобы отредактировать

Артём Соколов

Фронтенд-разработчик

ОБО МНЕ

Фронтенд-разработчик с опытом вёрстки адаптивных интерфейсов на чистых HTML, CSS и JavaScript. Внимателен к деталям, доступности и производительности. Ищу задачи, где ценят аккуратный код и продуманный пользовательский опыт.

ОПЫТ РАБОТЫ

Фронтенд-разработчик

2023 — настоящее время

Веб-студия «Пиксель»

Вёрстка лендингов и корпоративных сайтов, поддержка адаптивности, интеграция с CMS, оптимизация скорости загрузки.

Стажёр-верстальщик

2022 — 2023

ООО «ВебСтарт»

Создание страниц по макетам Figma, исправление кросс-браузерных

Рисунок 2 — Повторный рендер сохранённого резюме: телефон, фрагмент